Title: DSC680-T301 Applied Data Science (2271-1)

Professor: *Xu Ashton*

Date: 26 Sep 2026

Author: *Akhil Anand Thazhaniyil Muralidharan*

Description: Week3 Milestone 2: Code for Generating the images for white paper.

Purpose

This notebook reads three uploaded telecom data sources and creates evidence-based figures for the Milestone 2 white paper. The first dataset contains cellular signal, SNR, distance, attenuation, and call information. The second contains mobile upload and download speed tests by operator, technology, and telecom region. The third is the Telstra fault-severity dataset, where `train.csv` supplies the target and the event, resource, severity, and log files supply related operational features.

The notebook does not assume the three datasets can be row-level merged. Each source is analyzed independently because the identifiers, observation units, and collection contexts differ. The resulting illustrations provide complementary evidence about network quality, performance variation, and fault severity.

In [19]:
```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

DATA_DIR = next(
    (
        path
        for path in (Path.cwd() / "DataSets", Path.cwd() / "Week3" / "DataSe
        if path.is_dir()
    ),
    None,
)
if DATA_DIR is None:
    raise FileNotFoundError("Could not locate the DataSets folder.")

OUTPUT_DIR = DATA_DIR.parent / "white_paper_figures"
OUTPUT_DIR.mkdir(exist_ok=True)
```

```
plt.rcParams["figure.dpi"] = 120
plt.rcParams["savefig.dpi"] = 300
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 100)

def extract_number(series):
    return pd.to_numeric(
        series.astype(str).str.extract(r"(\d+)", expand=False), errors="coer
    )

def save_figure(filename):
    path = OUTPUT_DIR / filename
    plt.tight_layout()
    plt.savefig(path, bbox_inches="tight")
    print(f"Saved: {path}")
    plt.show()

csv_files = sorted(DATA_DIR.rglob("*.csv"))
print(f"Data directory: {DATA_DIR.resolve()}")
print(f"CSV files found: {len(csv_files)}")
```

```
Data directory: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDr
ive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/DataSets
CSV files found: 9
```

1. Discover and Load the Data

The analysis begins by recursively scanning `DataSets`, including its nested `telstra_dataSets` folder. Every CSV file is loaded into a dictionary keyed by its relative path. An inventory table confirms that each source was read and reports its dimensions, missing values, and exact duplicate rows.

In [20]:
```
datasets = {
    path.relative_to(DATA_DIR).as_posix(): pd.read_csv(path)
    for path in csv_files
}

inventory = pd.DataFrame(
    [
        {
            "file": name,
            "rows": len(frame),
            "columns": frame.shape[1],
            "missing_values": int(frame.isna().sum().sum()),
            "duplicate_rows": int(frame.duplicated().sum()),
        }
        for name, frame in datasets.items()
    ]
).sort_values("file", ignore_index=True)

display(inventory)
```

	file	rows	columns	missing_values	duplicate_ro
0	April_MySpeed_2021.csv	275078	6	709	982
1	cellular-network-performance-data.csv	463	11	0	
2	telstra_dataSets/event_type.csv	31170	2	0	
3	telstra_dataSets/log_feature.csv	58671	3	0	
4	telstra_dataSets/resource_type.csv	21076	2	0	
5	telstra_dataSets/sample_submission.csv	11171	4	0	
6	telstra_dataSets/severity_type.csv	18552	2	0	
7	telstra_dataSets/test.csv	11171	2	0	
8	telstra_dataSets/train.csv	7381	3	0	

2. Prepare the Analysis Tables

Each source has a different unit of observation, so the tables remain separate. Numeric fields are converted explicitly, labels such as `location 118` are reduced to their numeric codes, and only records missing variables required for a specific calculation are excluded. Exact repeats in the speed-test data are retained because the file provides no test identifier or timestamp with which to prove that repeated measurements are erroneous.

For Telstra, the one-to-many event, resource, severity, and log tables are aggregated to one row per `id` before joining to the labeled training data. This prevents joins from multiplying records and biasing the severity analysis.

In [21]:
```
cellular = datasets["cellular-network-performance-data.csv"].copy()
cellular["Timestamp"] = pd.to_datetime(cellular["Timestamp"], errors="coerce
cellular_numeric = [
    "Signal Strength (dBm)", "SNR", "Call Duration (s)",
    "Attenuation", "Distance to Tower (km)",
]
for column in cellular_numeric:
    cellular[column] = pd.to_numeric(cellular[column], errors="coerce")
cellular["potential_degradation"] = (
    (cellular["Signal Strength (dBm)"] < -100) | (cellular["SNR"] < 15)
)
cellular = cellular.dropna(
    subset=["Timestamp", "Signal Strength (dBm)", "SNR", "Distance to Tower
)

speed_tests = datasets["April_MySpeed_2021.csv"].copy()
speed_tests = speed_tests.rename(columns={"download": "direction"})
speed_tests["speed"] = pd.to_numeric(speed_tests["speed"], errors="coerce")
speed_tests["signal_strength"] = pd.to_numeric(
    speed_tests["signal_strength"], errors="coerce"
```

```
)
speed_tests["direction"] = speed_tests["direction"].astype(str).str.lower().
speed_tests["operator"] = speed_tests["operator"].astype(str).str.upper().st
speed_tests["technology"] = speed_tests["technology"].astype(str).str.upper(
speed_tests["lsa"] = speed_tests["lsa"].replace({"NA": np.nan})

train = datasets["telstra_dataSets/train.csv"].copy()
events = datasets["telstra_dataSets/event_type.csv"].copy()
resources = datasets["telstra_dataSets/resource_type.csv"].copy()
severity = datasets["telstra_dataSets/severity_type.csv"].copy()
logs = datasets["telstra_dataSets/log_feature.csv"].copy()

train["location_code"] = extract_number(train["location"]).astype("Int64")
events["event_code"] = extract_number(events["event_type"]).astype("Int64")
resources["resource_code"] = extract_number(resources["resource_type"]).asty
severity["severity_code"] = extract_number(severity["severity_type"]).astype
logs["log_code"] = extract_number(logs["log_feature"]).astype("Int64")
logs["volume"] = pd.to_numeric(logs["volume"], errors="coerce").fillna(0)

severity_agg = severity.groupby("id").agg(
    severity_type_count=("severity_code", "nunique"),
    severity_code_max=("severity_code", "max"),
)
event_agg = events.groupby("id").agg(event_type_count=("event_code", "nuniqu
resource_agg = resources.groupby("id").agg(
    resource_type_count=("resource_code", "nunique")
)
log_agg = logs.groupby("id").agg(
    log_feature_count=("log_code", "nunique"),
    total_log_volume=("volume", "sum"),
    max_log_volume=("volume", "max"),
    mean_log_volume=("volume", "mean"),
)

fault = (
    train.merge(severity_agg, on="id", how="left")
    .merge(event_agg, on="id", how="left")
    .merge(resource_agg, on="id", how="left")
    .merge(log_agg, on="id", how="left")
)
feature_cols = [
    "location_code", "severity_type_count", "severity_code_max",
    "event_type_count", "resource_type_count", "log_feature_count",
    "total_log_volume", "max_log_volume", "mean_log_volume",
]
fault[feature_cols] = fault[feature_cols].fillna(0)
telstra = fault

prepared_summary = pd.DataFrame(
    {
        "analysis_table": ["Cellular performance", "Mobile speed tests", "Te
        "rows_ready": [len(cellular), len(speed_tests), len(fault)],
        "columns_ready": [cellular.shape[1], speed_tests.shape[1], fault.sha
    }
)
display(prepared_summary)
```

	analysis_table	rows_ready	columns_ready
0	Cellular performance	462	12
1	Mobile speed tests	275078	6
2	Telstra labeled faults	7381	12

Descriptive and Data-Quality Checks

The following summaries verify the usable ranges of the numeric variables after conversion. They also quantify zero-speed tests and missing regional or signal values so those limitations remain visible rather than being hidden by later aggregation.

In [22]:
```
print("Cellular numeric summary")
display(cellular[cellular_numeric + ["potential_degradation"]].describe().ro

speed_quality = pd.Series(
    {
        "total_records": len(speed_tests),
        "missing_speed": int(speed_tests["speed"].isna().sum()),
        "zero_speed": int(speed_tests["speed"].eq(0).sum()),
        "missing_signal_strength": int(speed_tests["signal_strength"].isna()
        "missing_region": int(speed_tests["lsa"].isna().sum()),
    },
    name="record_count",
)
print("Mobile speed-test quality checks")
display(speed_quality.to_frame())
print("Mobile speed and signal summary")
display(speed_tests[["speed", "signal_strength"]].describe().round(2))
```

```
Cellular numeric summary
```

	Signal Strength (dBm)	SNR	Call Duration (s)	Attenuation	Distance to Tower (km)
count	462.00	462.00	462.00	462.00	462.00
mean	-85.00	19.83	894.25	5.46	5.06
std	14.33	5.58	529.91	3.50	2.96
min	-118.68	10.27	11.52	0.04	0.03
25%	-95.94	14.84	418.38	2.81	2.50
50%	-84.26	19.57	908.33	4.76	5.18
75%	-74.56	24.68	1360.69	7.58	7.69
max	-50.12	29.96	1795.18	14.94	9.98

```
Mobile speed-test quality checks
```

	record_count
total_records	275078
missing_speed	0
zero_speed	1443
missing_signal_strength	46828
missing_region	709

Mobile speed and signal summary

	speed	signal_strength
count	275078.00	228250.00
mean	8827.67	-80.28
std	15504.36	17.31
min	0.00	-112.00
25%	815.00	-96.00
50%	3047.00	-77.00
75%	8383.00	-65.00
max	135806.00	-50.00

3. Cellular Signal Quality

Signal strength and signal-to-noise ratio (SNR) are examined against distance and operating environment. Because dBm values closer to zero represent stronger signals, a negative relationship between distance and signal strength would indicate degradation farther from a tower. The paired panels show both the continuous distance relationship and environmental differences.

In [23]:
```
signal_distance_corr = cellular["Distance to Tower (km)"].corr(
    cellular["Signal Strength (dBm)"]
)
snr_by_environment = (
    cellular.groupby("Environment")["SNR"].median().sort_values(ascending=Fa
)

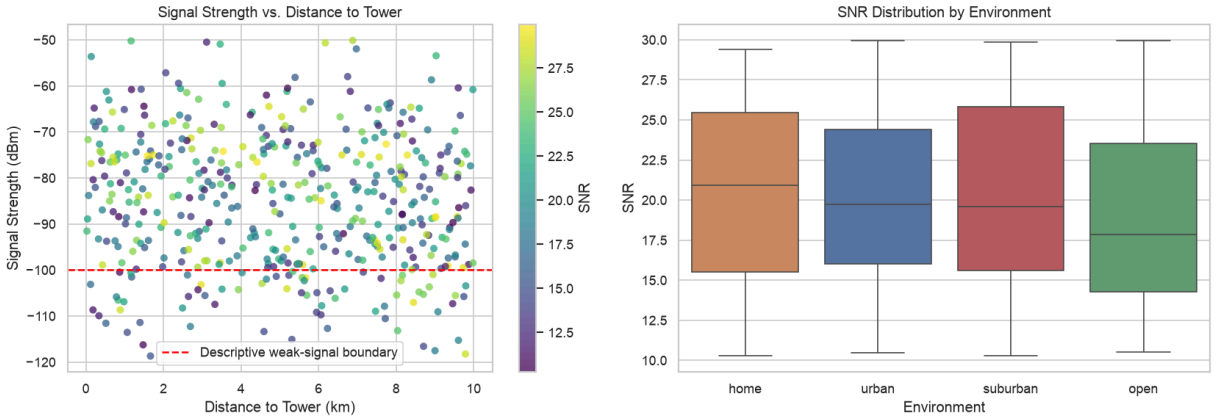
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
scatter = axes[0].scatter(
    cellular["Distance to Tower (km)"],
    cellular["Signal Strength (dBm)"],
    c=cellular["SNR"],
    cmap="viridis",
    alpha=0.75,
    edgecolor="none",
)
```

```
axes[0].axhline(
    -100, color="red", linestyle="--", linewidth=1.5,
    label="Descriptive weak-signal boundary",
)
fig.colorbar(scatter, ax=axes[0], label="SNR")
axes[0].set_xlabel("Distance to Tower (km)")
axes[0].set_ylabel("Signal Strength (dBm)")
axes[0].set_title("Signal Strength vs. Distance to Tower")
axes[0].legend()

sns.boxplot(
    data=cellular,
    x="Environment",
    y="SNR",
    hue="Environment",
    legend=False,
    order=snr_by_environment.index,
    ax=axes[1],
)
axes[1].set_title("SNR Distribution by Environment")
axes[1].set_xlabel("Environment")

save_figure("01_cellular_signal_quality.png")
print(f"Distance/signal correlation: {signal_distance_corr:.3f}")
print("Median SNR by environment:")
display(snr_by_environment.rename("median_snr").round(2).to_frame())
```

```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/01_cellular_signal_quality.png
```




```
Distance/signal correlation: -0.096
Median SNR by environment:
```

Environment	median_snr
home	20.90
urban	19.75
suburban	19.60
open	17.84

Cellular Degradation Checks

Following the reference analysis, a record is flagged as potentially degraded when signal strength is below -100 dBm or SNR is below 15. These transparent thresholds are exploratory rules rather than official service-level standards. The first figure compares the flag rate across environments; the second checks linear relationships among the measured cellular variables.

In [24]:
```
from matplotlib.ticker import PercentFormatter

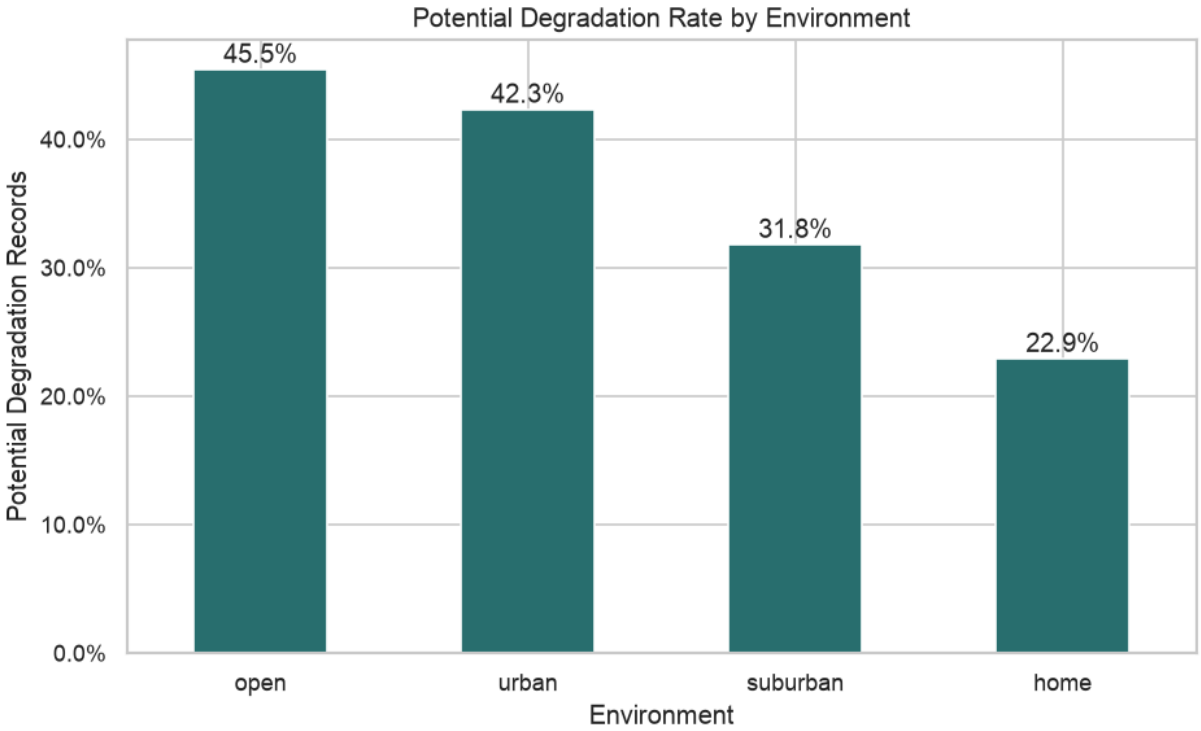
env_rate = (
    cellular.groupby("Environment")["potential_degradation"]
    .mean()
    .mul(100)
    .sort_values(ascending=False)
)

plt.figure(figsize=(8, 5))
ax = env_rate.plot(kind="bar", color="#287271")
ax.yaxis.set_major_formatter(PercentFormatter())
plt.xlabel("Environment")
plt.ylabel("Potential Degradation Records")
plt.title("Potential Degradation Rate by Environment")
plt.xticks(rotation=0)
for index, value in enumerate(env_rate):
    ax.text(index, value + 0.5, f"{value:.1f}%", ha="center")
save_figure("02_degradation_rate_by_environment.png")

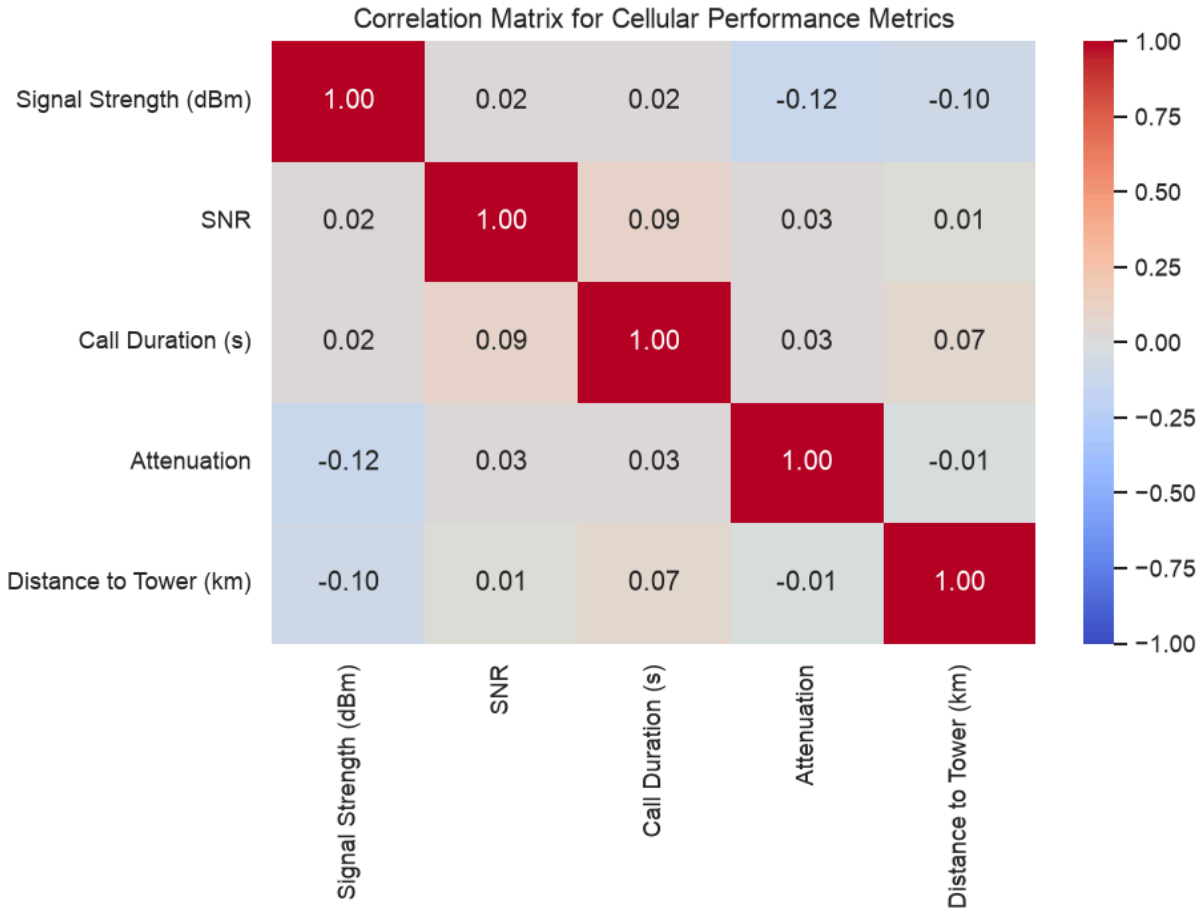
corr_cols = [
    "Signal Strength (dBm)", "SNR", "Call Duration (s)",
    "Attenuation", "Distance to Tower (km)",
]
plt.figure(figsize=(8, 6))
sns.heatmap(
    cellular[corr_cols].corr(), annot=True, fmt=".2f",
    cmap="coolwarm", center=0, vmin=-1, vmax=1,
)
plt.title("Correlation Matrix for Cellular Performance Metrics")
save_figure("03_cellular_correlation.png")

display(env_rate.rename("potential_degradation_percent").round(2).to_frame()
```

```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comca
st/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/02_degradat
ion_rate_by_environment.png
```




```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/03_cellular_correlation.png
```



Environment	potential_degradation_percent
open	45.45
urban	42.34
suburban	31.82
home	22.94

4. Mobile Speed-Test Performance

Mobile measurements are separated into upload and download directions. Median speed is used because the distribution is strongly right-skewed and includes extreme observations. Operator medians support provider comparison, while the regional chart ranks the 15 telecom service areas with the highest positive median download speed.

In [25]:
```
operator_speed = (
    speed_tests.dropna(subset=["speed"])
    .groupby(["operator", "direction"], as_index=False)["speed"]
    .median()
)

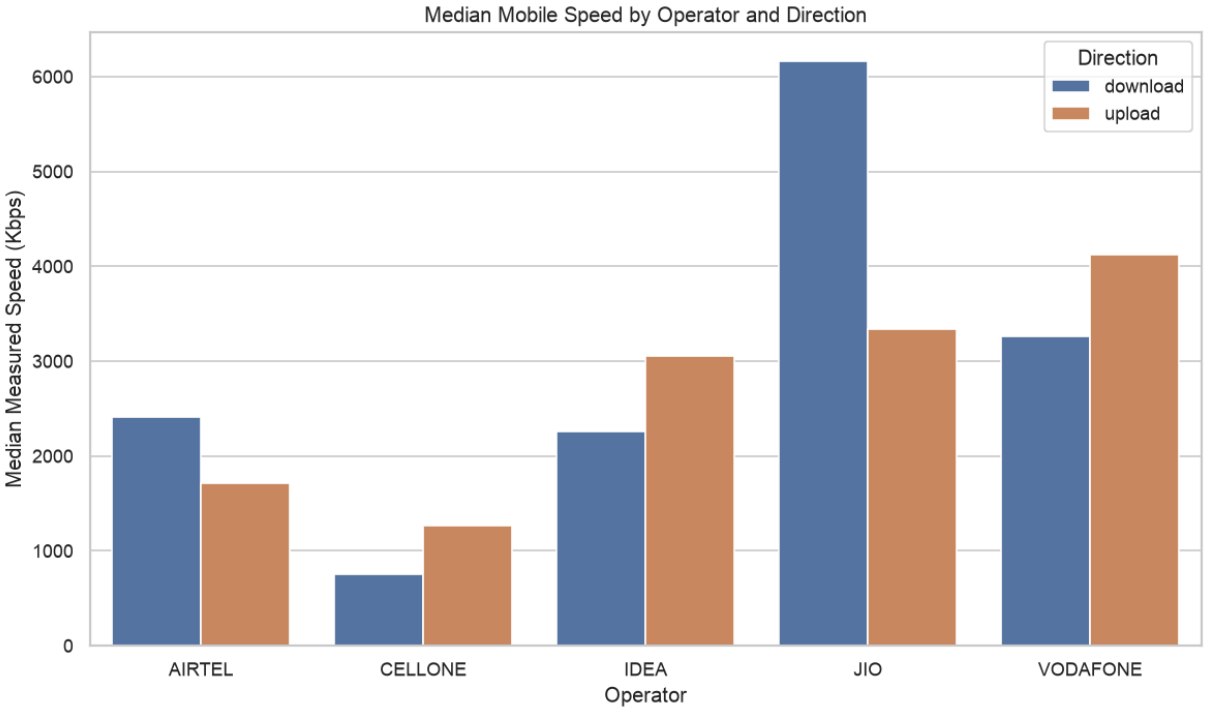
plt.figure(figsize=(10, 6))
sns.barplot(data=operator_speed, x="operator", y="speed", hue="direction")
plt.xlabel("Operator")
plt.ylabel("Median Measured Speed (Kbps)")
plt.title("Median Mobile Speed by Operator and Direction")
plt.legend(title="Direction")
save_figure("04_operator_median_speed.png")

regional_download = (
    speed_tests[
        (speed_tests["direction"] == "download")
        & speed_tests["lsa"].notna()
        & (speed_tests["speed"] > 0)
    ]
    .groupby("lsa")["speed"]
    .median()
    .sort_values(ascending=False)
    .head(15)
    .sort_values()
)

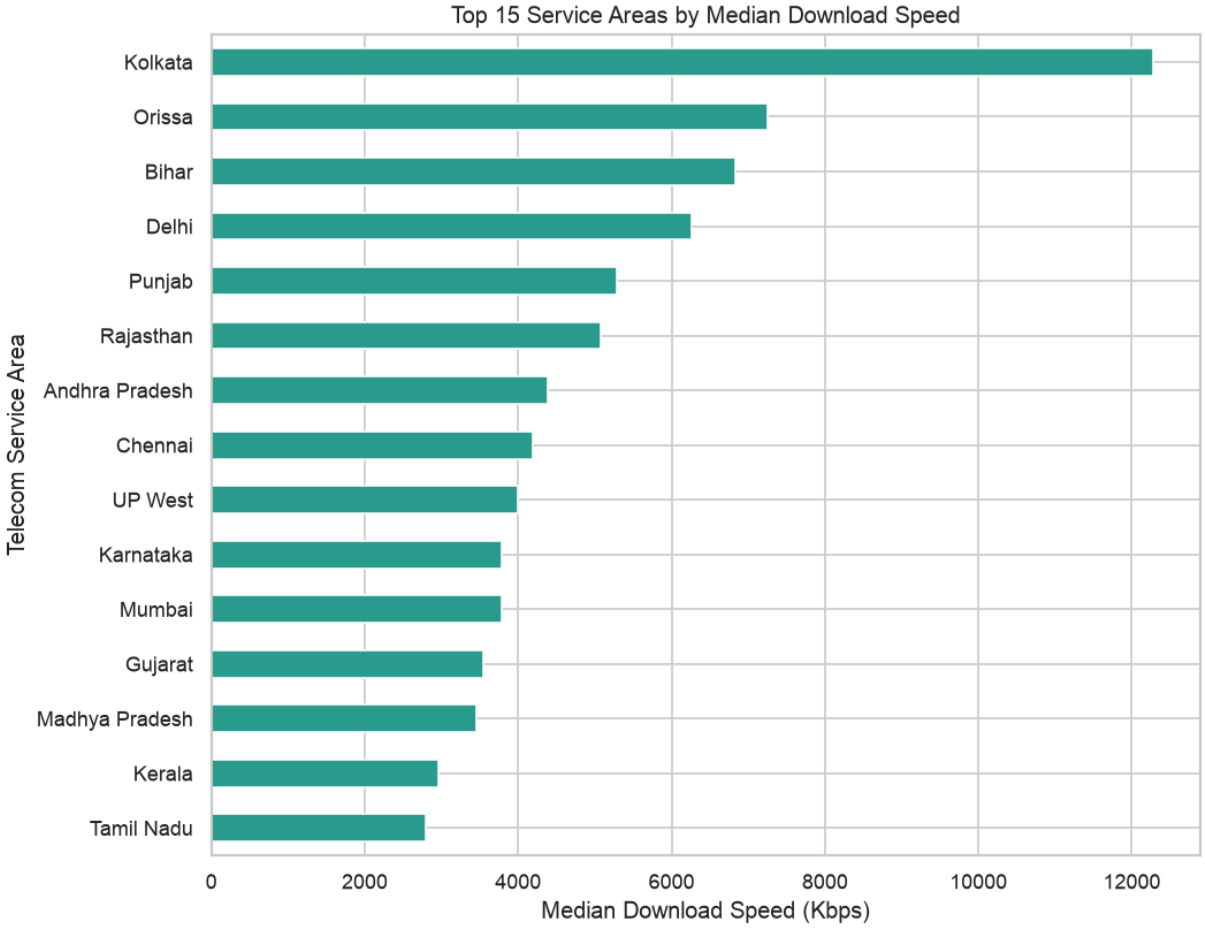
plt.figure(figsize=(9, 7))
regional_download.plot(kind="barh", color="#2A9D8F")
plt.xlabel("Median Download Speed (Kbps)")
plt.ylabel("Telecom Service Area")
plt.title("Top 15 Service Areas by Median Download Speed")
save_figure("05_regional_median_download.png")
```

```
display(operator_speed.sort_values(["direction", "speed"], ascending=[True,
```

```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/04_operator_median_speed.png
```




```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/05_regional_median_download.png
```



	operator	direction	speed
6	JIO	download	6158.0
8	VODAFONE	download	3267.0
0	AIRTEL	download	2415.0
4	IDEA	download	2260.0
2	CELLONE	download	754.5
9	VODAFONE	upload	4122.0
7	JIO	upload	3336.5
5	IDEA	upload	3055.0
1	AIRTEL	upload	1710.0
3	CELLONE	upload	1270.0

5. Telstra Fault-Severity Patterns

The target distribution is checked before modeling because class imbalance can make accuracy misleading. Operational log volume is then compared across severity classes

on a logarithmic scale, which reduces the visual effect of the highly skewed volume distribution.

In [26]:
```
severity_counts = train["fault_severity"].value_counts().sort_index()
severity_pct = severity_counts.div(severity_counts.sum()).mul(100)

plt.figure(figsize=(8, 5))
ax = severity_counts.plot(kind="bar", color=["#4C78A8", "#F58518", "#E45756"
plt.xlabel("Fault Severity Class")
plt.ylabel("Number of Records")
plt.title("Distribution of Telstra Fault-Severity Classes")
plt.xticks(rotation=0)
for index, value in enumerate(severity_counts):
    ax.text(index, value, f"{value:,}\n({severity_pct.iloc[index]:.1f}%)", h
save_figure("06_fault_severity_distribution.png")

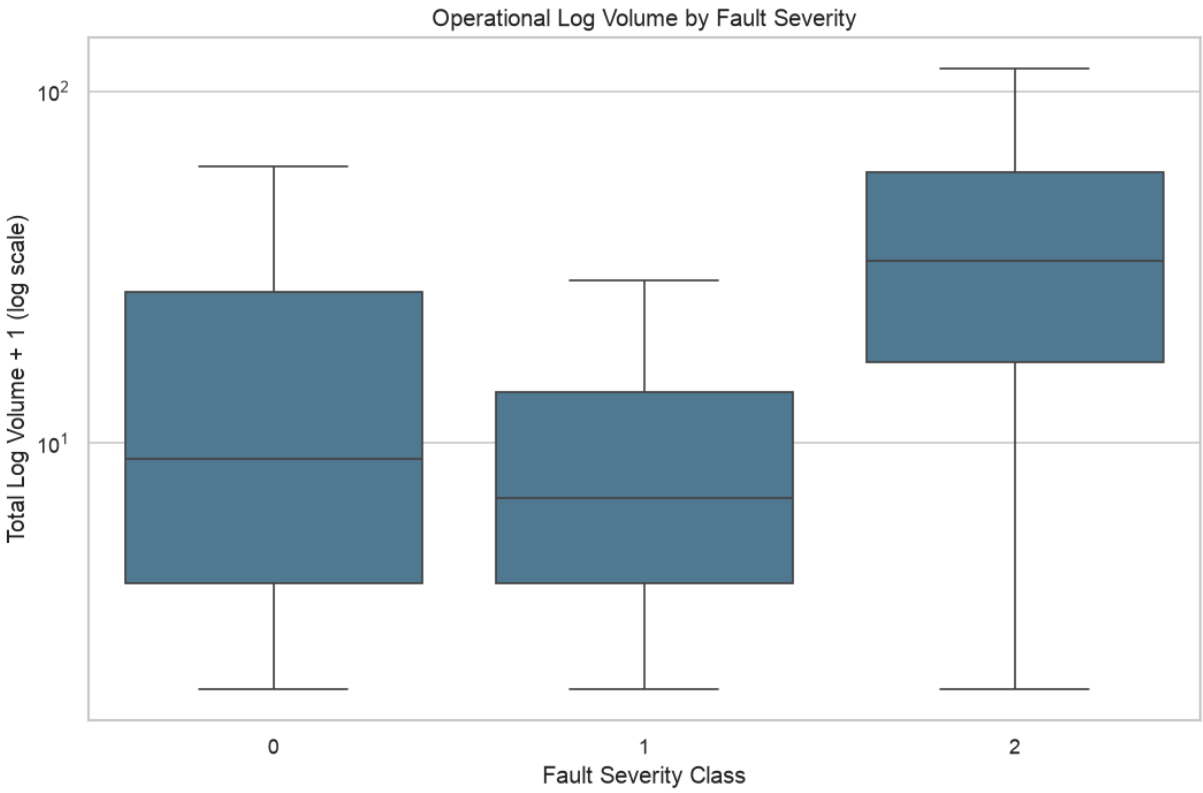
plot_fault = fault.copy()
plot_fault["total_log_volume_plot"] = plot_fault["total_log_volume"] + 1
plt.figure(figsize=(9, 6))
sns.boxplot(
    data=plot_fault,
    x="fault_severity",
    y="total_log_volume_plot",
    showfliers=False,
    color="#457B9D",
)
plt.yscale("log")
plt.xlabel("Fault Severity Class")
plt.ylabel("Total Log Volume + 1 (log scale)")
plt.title("Operational Log Volume by Fault Severity")
save_figure("07_log_volume_by_severity.png")

fault_summary = fault.groupby("fault_severity")[
    ["event_type_count", "resource_type_count", "log_feature_count", "total_
].median().round(2)
display(fault_summary)
```

```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comca
st/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/06_fault_se
verity_distribution.png
```




```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/07_log_volume_by_severity.png
```



fault_severity	event_type_count	resource_type_count	log_feature_count	total_log_vol
0	2.0	1.0	2.0	
1	2.0	1.0	2.0	
2	2.0	1.0	3.0	

6. Reproducible Baseline Model

A Random Forest baseline tests whether the aggregated operational features carry predictive information about fault severity. The 80/20 split is stratified to preserve class proportions, `class_weight="balanced"` reduces majority-class dominance, and `random_state=42` makes the result reproducible. The classification report and confusion matrix are emphasized because class-specific recall matters more than accuracy alone in this imbalanced problem.

In [27]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    classification_report,
    confusion_matrix,
)
from sklearn.model_selection import train_test_split

X = fault[feature_cols].astype(float)
y = fault["fault_severity"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = RandomForestClassifier(
    n_estimators=400,
    random_state=42,
    class_weight="balanced",
    min_samples_leaf=2,
    n_jobs=-1,
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
report = classification_report(y_test, predictions, digits=3, output_dict=Tr
print(classification_report(y_test, predictions, digits=3))

labels = sorted(y.unique())
matrix = confusion_matrix(y_test, predictions, labels=labels)
confusion_display = ConfusionMatrixDisplay(matrix, display_labels=labels)
confusion_display.plot(cmap="Blues", values_format="d")
plt.title("Random Forest Confusion Matrix")
save_figure("08_confusion_matrix.png")
```

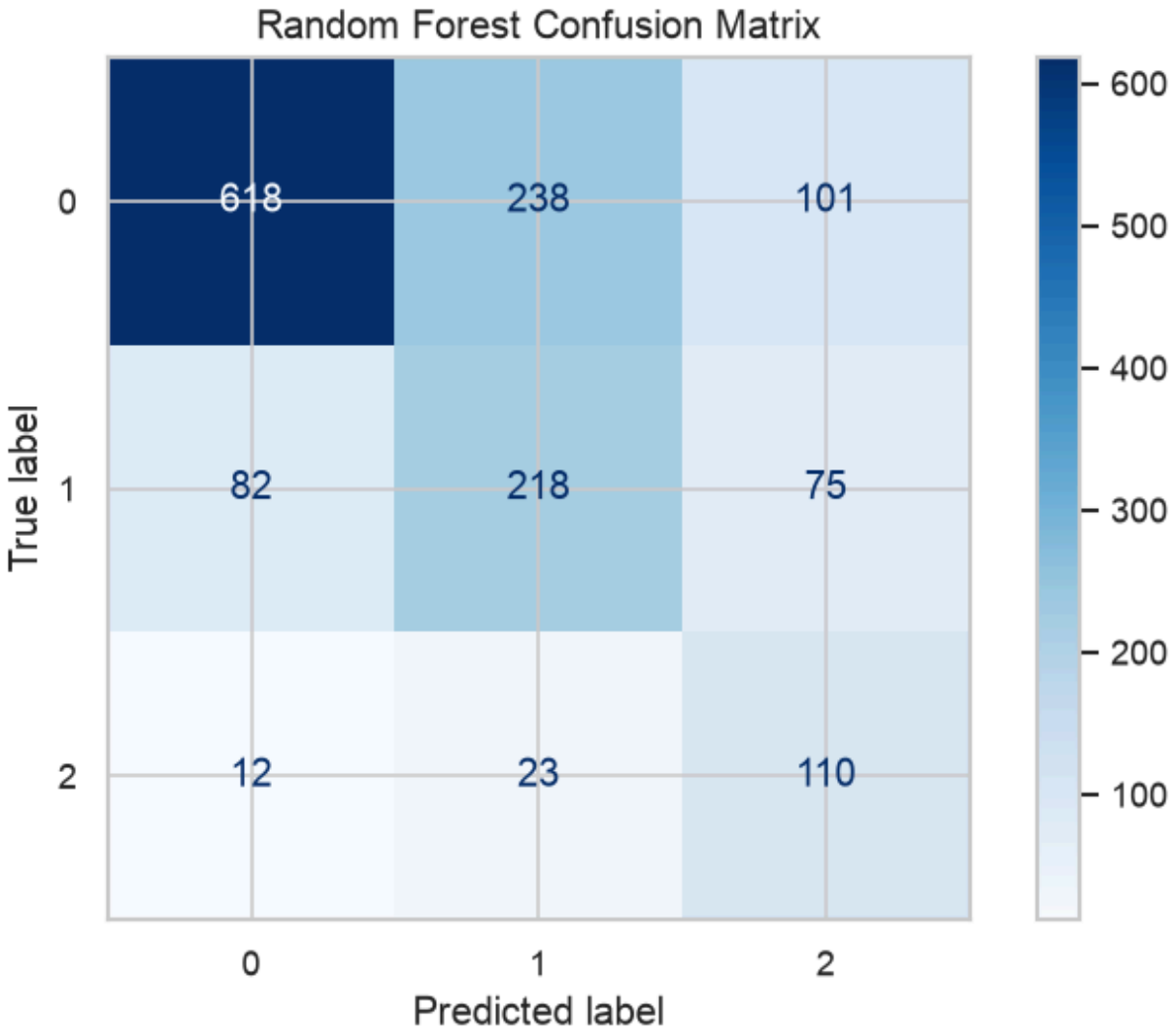
```
importance = pd.Series(
    model.feature_importances_, index=feature_cols, name="importance"
).sort_values()
plt.figure(figsize=(9, 6))
importance.plot(kind="barh", color="#6C5B7B")
plt.xlabel("Random Forest Feature Importance")
plt.ylabel("Feature")
plt.title("Baseline Feature Importance for Fault-Severity Prediction")
save_figure("09_feature_importance.png")
```

```
              precision    recall  f1-score   support

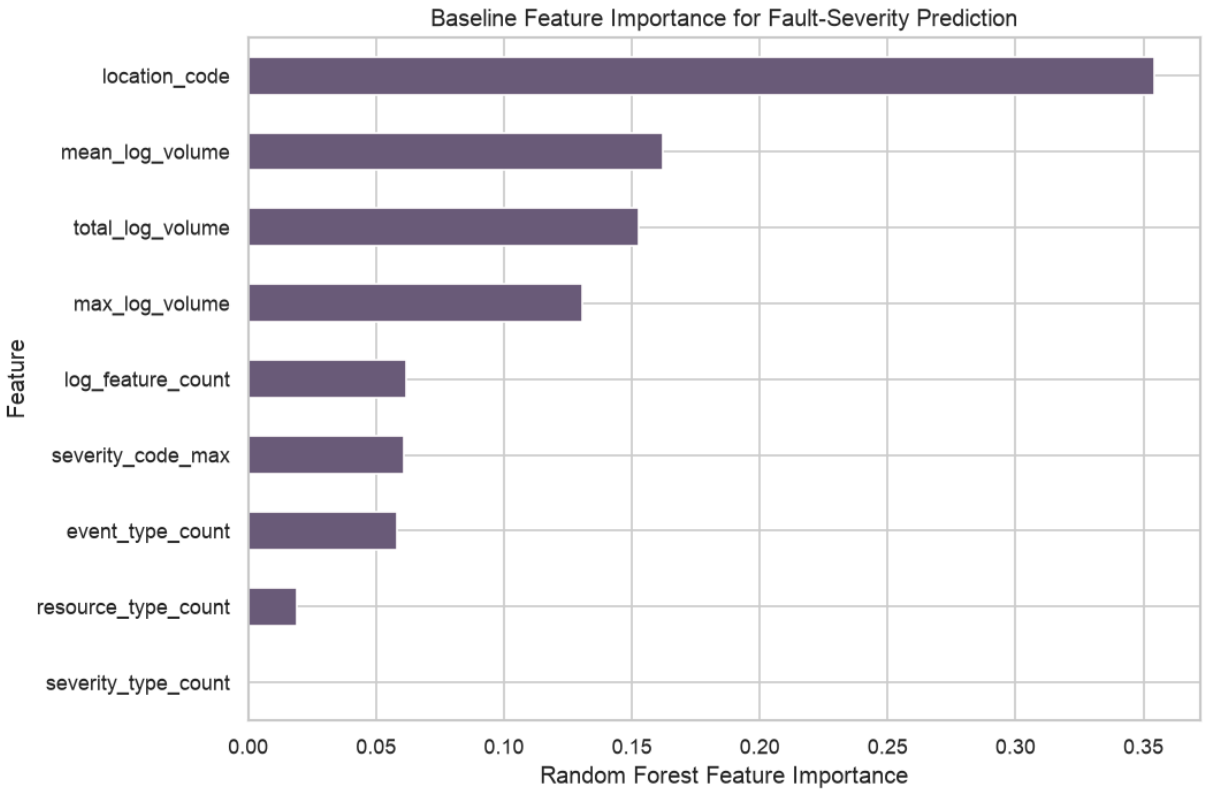
           0      0.868     0.646     0.741       957
           1      0.455     0.581     0.511       375
           2      0.385     0.759     0.510       145

    accuracy                          0.640      1477
   macro avg      0.569     0.662     0.587      1477
weighted avg      0.716     0.640     0.660      1477
```

```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/08_confusion_matrix.png
```



```
Saved: /Users/athazh341@apac.comcast.com/Library/CloudStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_paper_figures/09_feature_importance.png
```



7. Export Evidence Tables

The figures summarize the main patterns, while CSV exports preserve the exact values needed for a white-paper appendix or independent verification. Model importance measures predictive use within this baseline and must not be interpreted as causal influence.

In [28]:
```
classification_metrics = pd.DataFrame(report).T.rename_axis("class").reset_i
summary_tables = {
    "data_inventory.csv": inventory,
    "cellular_environment_degradation.csv": env_rate.rename(
        "potential_degradation_percent"
    ).reset_index(),
    "operator_median_speed.csv": operator_speed,
    "regional_median_download.csv": regional_download.rename(
        "median_download_speed"
    ).reset_index(),
    "fault_severity_distribution.csv": pd.DataFrame(
        {
            "fault_severity": severity_counts.index,
            "record_count": severity_counts.values,
            "percent": severity_pct.values,
        }
    ),
    "fault_operational_medians.csv": fault_summary.reset_index(),
```

```
    "baseline_classification_report.csv": classification_metrics,
    "baseline_feature_importance.csv": importance.sort_values(
        ascending=False
    ).rename_axis("feature").reset_index(),
}

for filename, table in summary_tables.items():
    table.to_csv(OUTPUT_DIR / filename, index=False)

print(f"Exported {len(summary_tables)} evidence tables to {OUTPUT_DIR.resolv
print(f"Top download region: {regional_download.idxmax()} ({regional_downloa
print(f"Top model feature: {importance.idxmax()} ({importance.max():.3f})")
print(f"Holdout accuracy: {report['accuracy']:.3f}")
print(f"Macro F1: {report['macro avg']['f1-score']:.3f}")
```

```
Exported 8 evidence tables to /Users/athazh341@apac.comcast.com/Library/Clou
dStorage/OneDrive-Comcast/AkhilAnandTM/BVU/DSC680/Assignments/Week3/white_pa
per_figures
Top download region: Kolkata (12,276 Kbps)
Top model feature: location_code (0.354)
Holdout accuracy: 0.640
Macro F1: 0.587
```

8. Findings and Limitations

Key Findings

1. **Distance alone is a weak linear predictor of signal strength.** The correlation between distance and signal strength is -0.096, indicating only a slight tendency toward weaker dBm readings at greater distances in this sample. Other propagation conditions therefore matter.
2. **The descriptive degradation rate varies by environment.** Open environments have the highest flagged rate at 45.45%, followed by urban at 42.34%, suburban at 31.82%, and home at 22.94%. Home also has the highest median SNR at 20.90, while open has the lowest at 17.84.
3. **Mobile performance differs materially by operator.** JIO has the highest median download result at 6,158 Kbps, while Vodafone has the highest median upload result at 4,122 Kbps. Kolkata leads the regional download ranking at 12,276 Kbps.
4. **Fault severity is imbalanced.** Class 0 accounts for 64.8% of labeled records, class 1 for 25.3%, and class 2 for 9.8%. This confirms that class-level precision, recall, and macro F1 are more informative than accuracy alone.
5. **Higher-severity faults show greater operational activity.** Median total log volume rises from 8 for class 0 and 6 for class 1 to 32 for class 2; the median log-feature count also rises from 2 to 3.
6. **The baseline detects the rarest class but remains imperfect.** Holdout accuracy is 0.640 and macro F1 is 0.587. Class 2 recall reaches 0.759, but precision is 0.385, indicating many false positives. Location code is the strongest model feature with importance 0.354.

Limitations

- The three source groups represent different populations and observation units and therefore must not be merged row by row.
- The degradation flag uses transparent exploratory thresholds, not an operator-approved service-level definition.
- The speed-test data has no unique test identifier or timestamp, so exact repeated rows are retained; 46,828 signal-strength values and 709 regional labels are missing.
- Regional and operator medians are descriptive and may reflect unequal sample sizes, device mix, geography, and self-selection in speed testing.
- The Random Forest is a reproducible baseline rather than a production model. Feature importance measures predictive use within this model and does not establish causality.

Recommended White-Paper Figures

Use the cellular signal-quality figure, degradation-rate chart, operator-speed comparison, fault-severity distribution, log-volume comparison, confusion matrix, and feature-importance chart as the main evidence. The correlation heatmap and regional ranking are useful supporting or appendix figures.